



UTEQ[®]
UNIVERSIDAD TECNOLÓGICA
DE QUERÉTARO

UNIVERSIDAD TECNOLÓGICA DE QUERÉTARO

Nombre del trabajo:

“SEGG-U1-P1”

Nombre de la materia:

Seguridad en el Desarrollo de Aplicaciones

Presenta:

OSCAR DANIEL MOTA HERNANDEZ

Profesor

Gerardo Ramírez Villarreal

Santiago de Querétaro, Qro. 19 de mayo 2026

1. Tabla de vulnerabilidades

Nombre	Descripción	Ataque	Impacto Potencial	Método de mitigación
A01	Control de acceso dañado Errores en la validación de permisos que posibilitan el acceso a datos o funciones sin autorización.	Un usuario común entra a la URL de admin: https://pagina.com/app/administrador Establecer un parámetro para observar los datos de otro usuario: <code>?acct=notmyacct</code>	* Ingreso no autorizado * Publicación de datos * Alteración o eliminación de información	• Negar de manera predeterminada • Confirmar permisos en el servidor • Aplicar el principio de privilegio mínimo
A02	Configuración de seguridad equivocada Configuraciones que no son seguras en servicios de nube, servidores y marcos	Entrar a la aplicación ejemplo sin suprimir: https://pagina.com/aplicacion/ejemplo Cuentas predeterminadas activadas con contraseña sin cambiar	• Acceso a información delicada • Presentación de información técnica • Asumir el control del servidor • Suprimir funciones innecesarias	• Suprimir las características innecesarias • Modificar las credenciales predeterminadas • Desactivar directorios en la web • Automatizar el endurecimiento
A03	Errores en la cadena de abastecimiento Debilidades en librerías, elementos y dependencias que están desactualizadas o tienen intenciones maliciosas.SolarWinds 2020: Actualización de	Debilidades en librerías, elementos y dependencias que están desactualizadas o tienen intenciones maliciosas.SolarWinds 2020: Actualización de seguridad dañina comprometió a 18,000 organizaciones	• Ejecución de código a distancia • Compromiso de millones de sistemas. • Infección por ransomware y malware	• Conservar la SBOM (la lista de componentes) • Realizar escaneos de CVEs de manera regular • Renovar los componentes a tiempo.

	seguridad dañina comprometió a 18,000 organizaciones	Log4Shell (CVE-2021-44228): RCE en Apache Log4j		• Utilizar fuentes oficiales
A04	Fallos Criptográficos Datos sensibles que no están encriptados, que tienen una mala encriptación o claves poco seguras.	• Base de datos que contiene contraseñas sin cifrar • Transmisión sin HTTPS en una conexión no segura • Algoritmos vulnerables como SHA1 o MD5	• Robo de credenciales • Pérdida de intimidad • Falta de cumplimiento del GDPR/PCI-DSS • Intercepción de datos	• Emplear TLS 1.2 o superior para la transmisión • Hash con Argon2 y bcrypt • Encriptar datos en reposo • Manejar claves en HSM
A05	Inyección Incorporación de código dañino en las consultas sin la validación de la entrada del usuario.	SQL Injection: ?id=' OR '1'='1 Command Injection: nslookup example.com; cat /etc/passwd	• Realización de código arbitrario • Acceso a la base de datos • Sustraer grandes cantidades de datos • Control del servidor	• Usar Prepared Statements • Validar y sanitizar entrada • Escapar caracteres sensibles • Usar ORMs seguros
A06	Problemas de diseño y arquitectura deficiente que suceden antes de la codificación. Incluye carencia de validaciones en procesos comerciales, lógica frágil y controles de seguridad. no presentes en el diseño original.	• Una cadena de cines tiene un máximo de 15 asientos disponibles para reservar sin necesidad de depósito. • Un asaltante elabora un ataque que busca reservar simultáneamente 600 sillas en varios cines, lo que provoca una pérdida significativa de ingresos.	• Pérdida de ingresos a causa del aprovechamiento de la lógica empresarial. • Fraude a través de sistemas que carecen de validaciones apropiadas • Adquisiciones fraudulentas automatizadas • Inconsistencias en los flujos de datos esenciales	• Aplicar el modelado de amenazas en la etapa de diseño • Crear historias de usuario que incluyan ejemplos de uso y de mal uso. • Hacer uso de bibliotecas de patrones de diseño seguro

A07	Deficiencias en los procesos de autenticación y administración de sesiones que posibilitan a atacantes que se hacen pasar por usuarios auténticos. Incorpora contraseñas poco seguras, ausencia de sesiones sin validación y multifactor.	• Sesiones que no se invalidaron de manera apropiada: El usuario cierra el navegador sin hacer clic. • Cerrar sesión. Si la sesión no ha caducado, otro usuario que esté en el mismo ordenador puede entrar a su cuenta. • Dificultad frecuente en el inicio de sesión único (SSO).	• Acceso no autorizado a las cuentas de los usuarios • Suplantación de identidad. • Hurto de datos financieros y personales	• Poner en práctica la autenticación multifactor (MFA) en todas las plataformas relevantes • Emplear administradores de contraseñas para robustecer la creación de contraseñas • NO emplear contraseñas preestablecidas o frágiles
A08	Ausencia de verificación del estado de integridad en las actualizaciones de software, datos serializados y código que ha sido descargado. Sistema tiene confianza en software/datos sin comprobar si han sido manipulados o vienen de una fuente confiable.	Actualización del firmware sin firma: Router doméstico es actualizado sin firmar de manera digital. El atacante intercepta y altera el firmware antes de que se instale, obteniendo así un control permanente del aparato.	• Ejecución de código malicioso en servidores o dispositivos • Compromiso de un sistema entero a través del firmware alterado • Sustracción de datos y credenciales de los usuarios	• Emplear firmas digitales para comprobar la integridad de los datos y del software • Únicamente descargar librerías o dependencias de repositorios oficiales confiables. • Llevar a cabo un procedimiento de revisión del código para modificaciones y configuraciones.

A09	Carencia de sistemas de alerta y falta de un registro apropiado de los sucesos relacionados con la seguridad. Los ataques pueden permanecer indetectados durante años sin la existencia de monitoreo y registros.	Una compañía aérea europea experimentó una fuga de 400,000 datos de pago debido a las fallas en su aplicación de pagos. No se percató del ataque sin alertas. GDPR le impuso una multa de 20 millones de libras esterlinas.	• Ataques que no son detectados durante meses o años • Falta de capacidad para reaccionar con rapidez a los incidentes • Pérdida de datos forenses digitales para la investigación	• Salvaguardar la integridad de los registros frente a la alteración (base de datos de solo escritura) • Examinar los registros en busca de conductas sospechosas • Realizar copias de seguridad de los registros (no guardar únicamente a nivel local)
A10	Gestión ineficaz de excepciones, errores y situaciones imprevistas. Implementación no evita, identifica ni reacciona apropiadamente a circunstancias anormales, lo que da lugar a sistema en una condición de imprevisibilidad.	Divulgación de datos privados: La aplicación tiene un manejo deficiente del error de la base de datos. y presenta al usuario el mensaje de error en su totalidad. Más errores son causados por el atacante. para adquirir datos del sistema y mejorar el ataque de inyección SQL.	• Denegación de servicio (DoS) por falta de recursos • Presentación de datos técnicos confidenciales a través de mensajes de error • Deterioro de operaciones financieras esenciales	• Planificar para lo peor: Prepárese para situaciones excepcionales. • Identificar y gestionar de inmediato los errores en su lugar de ocurrencia • Desarrollar una gestión global de excepciones centralizada

2. Casos reales de vulnerabilidades

A01

Caso 1: MOVEit Transfer (CVE-2023-34362)

Empresa/Producto:	Progress Software (MOVEit Transfer)
Año:	2023
Cómo fue explotada:	El grupo de ransomware Cl0p aprovechó una vulnerabilidad grave en el control de acceso. Un endpoint esencial no tenía la autenticación apropiada, lo que dejaba entrar a atacantes sin credenciales válidas. Pese a que el problema técnico subyacente fue SQL Injection, la ausencia de verificación del acceso fue el verdadero inconveniente. Esto llevó al compromiso de más de 2,000 organizaciones y a la revelación de datos de más de 62 millones de personas.

Caso 2: Kia y Hyundai Connected Vehicles API (2024)

Empresa/Producto:	Kia y Hyundai Motors
Año:	2024
Cómo fue explotada:	Investigadores hallaron errores graves de control de acceso en las APIs de los autos conectados. Los atacantes podían obtener acceso a APIs creadas para usuarios autenticados solo con proporcionar un número de VIN o de matrícula, lo que daba la posibilidad de controlar funciones del automóvil a distancia. Las vulnerabilidades surgieron de una verificación de autorización insuficiente y una autenticación débil,

A02

Caso 1: Configuración de Permisos S3 en AWS Mal Configurados (2023-2024)

Empresa/Producto:	Múltiples empresas
Año:	2023-2024
Cómo fue explotada:	Debido a los buckets S3 de AWS con permisos por defecto públicos, infinidad de compañías sufrieron filtraciones de datos. Los atacantes tenían la capacidad de listar los directorios, descargar archivos delicados y obtener acceso a las credenciales. Entre los ejemplos se encuentran empresas de Fortune 500, nuevas empresas tecnológicas y agencias de salud que tuvieron datos médicos, credenciales de empleados y contratos expuestos a la vista pública.

Caso 2: Aplicaciones de Ejemplo sin Eliminar (2023)

Empresa/Producto:	Proveedores genéricos de aplicaciones
Año:	2023
Cómo fue explotada:	Numerosos servidores de aplicaciones vienen con programas de ejemplo que tienen vulnerabilidades de seguridad ya identificadas. Si no se las retira de producción, los atacantes tienen la posibilidad de acceder a estas aplicaciones, emplear cuentas de administrador predeterminadas y tomar control absoluto del sistema. Esto ha tenido un impacto en compañías que no llevaron a cabo las adecuadas limpiezas durante el despliegue.

A03

Caso 1: Ataque a SolarWinds (2019-2020)

Empresa/Producto:	SolarWinds Orion Platform
Año:	2020
Cómo fue explotada:	SolarWinds, un proveedor de software confiable, sufrió un compromiso y sus actualizaciones de software se vieron contaminadas con código dañino. Cerca de 18,000 entidades, incluidas las agencias del gobierno estadounidense, descargaron e instalaron la actualización comprometida. Esta fue una de las más grandes brechas en la cadena de suministro, lo que permitió el espionaje en términos de infraestructura crítica.

Caso 2: Shai-Hulud npm Worm (2025)

Empresa/Producto:	npm Registry (miles de desarrolladores)
Año:	2025
Cómo fue explotada:	La cadena de suministro JavaScript fue el primer blanco del gusano npmautopropagante que tuvo éxito. Se han publicado versiones maliciosas de paquetes npm populares que, al ejecutarse scripts después de la instalación, sustraían información confidencial. Los atacantes identificaban los tokens npm en las computadoras de los desarrolladores y los empleaban para propagar versiones maliciosas de cualquier paquete disponible. El malware se extendió a más de quinientas versiones de paquetes antes de ser descubierto y detenido por npm.

A04

Caso 1: Target Corporation (2013) - Almacenamiento débil de contraseñas

Empresa/Producto:	Target Corporation
Año:	2013
Cómo fue explotada:	La conocida brecha de Target comprometió 40 millones de registros de tarjetas de crédito. A pesar de que el primer ataque fue de inyección SQL, los datos no se encontraban correctamente cifrados. Los atacantes entraron a los números de las tarjetas de crédito sin encriptar, lo que posibilitó el hurto y el fraude a gran escala. Esta brecha produjo perjuicios de \$18.5 millones y un notable daño a la reputación.

Caso 2: Adobe Systems (2013) - Contraseñas con Hash débil (MD5)

Empresa/Producto:	Adobe Systems
Año:	2013
Cómo fue explotada:	Adobe sufrió un ataque informático y se expusieron 150 millones de registros de usuarios. Se suponía que las contraseñas estaban "encriptadas", pero en realidad se empleaban XOR simples y algunos hashes MD5 poco robustos. Los atacantes lograron descodificar millones de contraseñas utilizando tablas arcoíris. A pesar de que había datos encriptados, la encriptación era tan débil que resultó prácticamente ineficaz frente a ataques contemporáneos.

A05

Caso 1: BeyondTrust PostgreSQL Injection (Enero 2025)

Empresa/Producto:	BeyondTrust Remote Support Platform
Año:	2025
Cómo fue explotada:	Para acceder al sistema de BeyondTrust, los atacantes aprovecharon una vulnerabilidad de SQL Injection en PostgreSQL (CVE-2025-1094). El Departamento del Tesoro de Estados Unidos fue alcanzado por la cadena de intrusión. El problema consistió en un manejo incorrecto de la entrada del usuario en las consultas a la base de datos, una clase de error que se había documentado más de 20 años atrás. La vulnerabilidad posibilitó que información confidencial del gobierno fuera accedida sin autorización.

Caso 2: WordPress Plugins XSS Masivo (2024)

Empresa/Producto:	Múltiples plugins de WordPress
Año:	2024
Cómo fue explotada:	Se hallaron varias vulnerabilidades XSS almacenadas en plugins de WordPress muy utilizados (CVE-2023-40000, CVE-2024-2194, CVE-2023-6961). Se hizo un uso activo del plugin WP Statistics, que cuenta con 600,000 instalaciones activas. Los atacantes lograban inyectar JavaScript perjudicial en campos no sanitizados, lo que les permitía robar sesiones de administrador, introducir puertas traseras y comprometer miles de páginas web.

A06

Caso 1: Especulación Bot de Tarjetas Gráficas (2020-2021)

Empresa/Producto:	Minoristas de electrónica (sin protección anti-bot)
Año:	2021
Cómo fue explotada:	Los especuladores emplearon robots para adquirir automáticamente todas las tarjetas gráficas de la serie RTX 30 que estaban disponibles, en medio de la falta de estos productos. El problema fue el mal diseño: no había límites de compra por usuario, no existía CAPTCHA y no se detectaban bots. Esto provocó que los productores fueran mal vistos, que los clientes legítimos se frustraran y que hubiera una gran pérdida de ingresos. El sistema no fue planeado con esta vía de ataque en mente.

Caso 2: Lógica de Negocio Débil en Reservas de Cines

Empresa/Producto:	Cadenas de cines
Año:	2022-2023
Cómo fue explotada:	Los sistemas de reservas en los cines posibilitaban la compra de más de 600 asientos simultáneamente, eludiendo las restricciones de 15 butacas fijadas para "reservas sin depósito". Los atacantes moldearon el flujo comercial y descubrieron que las transacciones no tenían validaciones apropiadas. El diseño inseguro posibilitó un fraude a gran escala, lo que resultó en la pérdida de cientos de miles de dólares sin que se descubriera la validación de los cambios de estado.

A07

Caso 1: Ataque Credential Stuffing a LinkedIn (2023-2024)

Empresa/Producto:	LinkedIn (durante ataques globales)
Año:	2024
Cómo fue explotada:	LinkedIn sufrió de manera masiva ataques de credential stuffing cuando los atacantes adquirieron listas de usuario/contraseña a través de brechas anteriores. A pesar de que LinkedIn implementó defensas, se accedió a millones de cuentas mediante credenciales comprometidas. La información laboral delicada y las conexiones profesionales se hicieron accesibles porque el ataque demostró que numerosos usuarios reutilizan sus contraseñas en diferentes plataformas.

Caso 2: Sesiones sin Expiración en Aplicación SSO (2023)

Empresa/Producto:	Proveedores genéricos SSO
Año:	2023
Cómo fue explotada:	Después de que los usuarios cerraban sesión en muchos sistemas Single Sign-On (SSO) corporativos, las sesiones no se invalidaban correctamente. Los usuarios cerraban los navegadores sin hacer clic en la opción de cierre de sesión, pero su sesión seguía activa. Si alguien más entraba en la misma computadora, tenía acceso a todas las aplicaciones (como los sistemas de pagos, los documentos y el correo electrónico) que estaban autenticadas. En empresas que compartían recursos, este fue un problema crucial.

A08

Caso 1: Actualización de Firmware sin Firma en Routers

Empresa/Producto:	Múltiples fabricantes de routers domésticos
Año:	2022-2024
Cómo fue explotada:	Los dispositivos IoT, los routers de casa y los decodificadores de cable no revisaban las actualizaciones del firmware a través de firmas digitales. Los atacantes en redes locales eran capaces de interceptar y alterar el firmware antes de su instalación, lo que les permitía obtener control permanente sobre el equipo. Los usuarios estaban expuestos hasta que el dispositivo se volviera obsoleto, sin ningún sistema para resolverlo.

Caso 2: Deserialización Insegura en Java (2023)

Empresa/Producto:	Aplicaciones genéricas con microservicios
Año:	2023
Cómo fue explotada:	Se violaron aplicaciones que transmitían el estado de usuario serializado entre microservicios sin comprobar la integridad. Los atacantes identificaban la firma del objeto Java "rOO" en base64 y manipulaban el objeto serializado con el fin de ejecutar un código remoto arbitrario en el servidor. El inconveniente consistía en confiar en la información del cliente sin comprobar que no había sido alterada.

A09

Caso 1: Proveedor de Seguros Médicos (7+ años sin detectar)

Empresa/Producto:	Proveedor de seguros médicos infantiles (anónimo)
Año:	2013-2020
Cómo fue explotada:	Un proveedor de seguros de salud sufrió un ataque cibernético durante más de siete años sin ser descubierto. Hacker lograron acceder y alterar datos médicos de 3.5 millones de niños. La brecha fue descubierta por terceros. El motivo: No se contaba con monitoreo ni registros. Sin una auditoría, no se pudo identificar que el sistema había sido vulnerado durante un periodo largo de tiempo. Las consecuencias en términos de privacidad fueron desastrosas.

Caso 2: Aerolínea Europea - Multa GDPR por Negligencia (2024)

Empresa/Producto:	Aerolínea europea mayor
Año:	2024
Cómo fue explotada:	Una compañía aérea se vio afectada por una filtración de 400,000 registros de pagos de clientes debido a las vulnerabilidades en su sistema de pagos. No pudo detectar el ataque a tiempo debido a la falta de registros apropiados. Debido a la ausencia de vigilancia y registro de seguridad, fue sancionada con 20 millones de libras esterlinas bajo GDPR. Una de las más grandes multas por violación de la privacidad se debió a la negligencia en la detección de incidentes.

A10

Caso 1: Exposición de Base de Datos vía Mensajes de Error (2023)

Empresa/Producto:	Aplicaciones genéricas
Año:	2023
Cómo fue explotada:	Una aplicación web exhibía mensajes de error con gran detalle, los cuales revelaban la estructura de la base de datos y las versiones de los elementos. Los atacantes causaban deliberadamente errores con el propósito de obtener información del sistema, y luego perfeccionaban sus ataques de inyección SQL. El mal manejo de excepciones se volvió un vector de reconocimiento, lo que permitió ataques más certeros.

Caso 2: DoS por Agotamiento de Recursos (Fugas de Memoria)

Empresa/Producto:	Aplicaciones con manejo pobre de excepciones
Año:	2024
Cómo fue explotada:	Las aplicaciones que identificaban excepciones al cargar archivos, pero no liberaban los recursos de forma apropiada, experimentaban un agotamiento de memoria. Los atacantes enviaban miles de archivos malformados, cada uno de los cuales dejaba recursos bloqueados, hasta que la memoria disponible se terminaba y se producía una Denegación de Servicio. El problema fue el manejo erróneo de las excepciones, sin la limpieza de recursos.

3. Ejemplo de código seguro y código no seguro

A01: CONTROL DE ACCESO MAL

CÓDIGO VULNERABLE

```
// Cualquier usuario ve datos de otros
app.get('/user/:id', (req, res) => {
  db.query('SELECT * FROM users WHERE id = ?', [req.params.id],
    (err, user) => res.json(user));
});

// Un usuario accede a /user/2 y ve TODO de otro usuario
```

CÓDIGO SEGURO

```
// Solo ve sus propios datos
app.get('/user/:id', (req, res) => {
  if (req.params.id !== req.user.id) {
    return res.status(403).send('No permitido');
  }
  db.query('SELECT * FROM users WHERE id = ?', [req.params.id],
    (err, user) => res.json(user));
});

// Ahora: /user/2 solo funciona si ERES usuario 2
```

A05: INYECCIÓN SQL

CÓDIGO VULNERABLE

```
// Usuario puede ejecutar código SQL
const username = req.body.username; // "admin' --"
const query = "SELECT * FROM users WHERE username = '" + username + "'";
// Resultado: SELECT * FROM users WHERE username = 'admin' --'
// Esto accede como admin SIN contraseña
```

CÓDIGO SEGURO

```
// Usuario solo puede enviar datos
const username = req.body.username; // "admin' --"
const query = "SELECT * FROM users WHERE username = ?";
db.query(query, [username]);
// Ahora "admin' --" es TEXTO, no código SQL
```


// El ? es un placeholder que protege

A07: FALLOS DE AUTENTICACIÓN

CÓDIGO VULNERABLE

// Guardar sin hash

```
app.post('/register', (req, res) => {  
  const password = req.body.password; // "123456"  
  db.query('INSERT INTO users (password) VALUES (?)', [password]);  
  // Si la BD se hackea: todas las contraseñas en texto plano  
});
```

CÓDIGO SEGURO

// Guardar con bcrypt

```
const bcrypt = require('bcrypt');  
  
app.post('/register', async (req, res) => {  
  const password = req.body.password; // "123456"  
  const hashed = await bcrypt.hash(password, 10);  
  db.query('INSERT INTO users (password) VALUES (?)', [hashed]);  
  // Ahora: $2b$10$xxxx (hash con salt automático)  
  // Si se hackea: solo ve hashes, no contraseñas  
});
```

A03: CROSS-SITE SCRIPTING (XSS)

CÓDIGO VULNERABLE

// Mostrar comentario directamente

```
const comment = req.body.text; // "<script>alert('hacked')</script>"  
res.send("<p>" + comment + "</p>");  
// El navegador ejecuta el script (XSS)
```

CÓDIGO SEGURO

// Convertir < y > a entidades HTML

```
function escape(text) {  
  return text.replace(/</g, '&lt;').replace(/>/g, '&gt;');  
}
```

```
const comment = req.body.text;  
res.send("<p>" + escape(comment) + "</p>");  
// Ahora: &lt;script&gt; (texto, no código)
```

4. Ranking personal

#1 A05: INYECCIÓN (SQL + XSS)

- **Justificación:** 62,445 CVEs (la más utilizada). Acceso directo a la base de datos, con el robo de millones de registros. Si no hay registros, una única inyección puede comprometer datos de manera masiva sin dejar huellas.

#2 A01: CONTROL DE ACCESO AVERIADO

- **Justificación:** Impacta todas las aplicaciones. Altera los parámetros en la URL para tener acceso a las funciones de administración y a los datos de otros usuarios. No necesita cambios en el código, solo "adivinar" las URL.

#3 A07: FALLOS DE AUTENTICACIÓN

- **Justificación:** El credential stuffing y el password spraying ponen en peligro anualmente millones de cuentas. El atacante se hace pasar por el usuario en su totalidad, obteniendo acceso inmediato y completo.

#4 A09: REGISTRO Y ALERTAS

- **Justificación:** Sin logs, los ataques pueden pasar desapercibidos durante años (en un caso real: más de 7 años). Aunque no causa daño directo, posibilita que otros ataques persistan indefinidamente = daño exponencial.

#5 A04: FALLOS CRIPTOGRÁFICOS

- **Justificación:** Se producen robos de información sin encriptar en brechas masivas. Se ha causado un daño permanente (contraseñas, números de tarjeta). Sin embargo, HTTPS reduce muchos casos, y las contraseñas débiles son un error del usuario.

5. Conclusión

Para todo desarrollador profesional, es crucial conocer el OWASP Top 10. No es una opción, sino una necesidad vital para el desarrollo de software seguro. Mediante este estudio, hemos observado que brechas masivas que impactan a millones de usuarios y producen pérdidas financieras enormes son consecuencia de vulnerabilidades como el Control de Acceso Averiado, la inyección SQL y el fallo de autenticación.

Las estadísticas son claras: el 100% de las aplicaciones analizadas presentan vulnerabilidades OWASP, y casos reales como Target (40 millones de tarjetas), Adobe (150 millones de usuarios) y SolarWinds (18,000 organizaciones) demuestran que estas no son amenazas teóricas, sino realidades cotidianas. Es casi seguro que se cometen estos errores si no se tiene conocimiento de OWASP Top 10.

En mi trabajo, utilizaré este conocimiento en múltiples niveles. En el diseño, desde el comienzo implementaría arquitectura segura y análisis de amenazas. En desarrollo, usaría prepared statements contra SQL Injection, validará autorización en cada endpoint y aplicaría bcrypt para contraseñas. En las revisiones de código, me enfocaría en buscar patrones OWASP. En testing, llevaría a cabo pruebas de seguridad automatizadas. En la supervisión, generaría registros y advertencias para identificar ataques.

La verdad es que la seguridad no es un simple cheque final, sino un compromiso continuo. Cada línea de código que redactó tiene un impacto en la privacidad y seguridad de los usuarios reales. El OWASP Top 10 me da la brújula para salvaguardar dicha confianza, y asumo ese deber profesional con total seriedad.